
Ejercicios Dictionarios

Ejercicios Guía

Hecho por: Matías González



Índice

Índice.....	2
GitHub.....	3
– Cómo usar Github (“ El Método en Duoc”) –.....	3
Ejercicio 1: Administración de Propietarios (Fácil).....	7
– Contexto –.....	7
– Objetivos –.....	7
– Resultado Esperado –.....	8
Ejercicio 2: Pokédex de Movimientos (Medio).....	9
– Contexto –.....	9
– Objetivos –.....	9
Ejercicio 3: Registro de Videojuegos (Medio).....	11
– Objetivos –.....	11
– Consideraciones –.....	11
– El sistema deberá validar–.....	11
– Ejemplos Consola –.....	12
Ejercicio 4: Team Builder de LoL (Difícil).....	13
– Contexto –.....	13
– Objetivos –.....	14
– Consideraciones –.....	14
– El sistema deberá validar –.....	15
– Bonus: MVP –.....	15
– Ejemplos Consola –.....	15
Ejercicio 5: Registro Planetario (Muy Difícil).....	17
– Contexto –.....	17
– Objetivos –.....	19
– Consideraciones –.....	19
– El sistema deberá validar–.....	20
– Ejemplos Consola –.....	20
¡Ánimo Chicos!.....	23

GitHub

– Cómo usar Github (“El Método en Duoc”) –

Antes de comenzar con los ejercicios, es necesario preparar un repositorio donde se almacenará y respaldará el trabajo realizado durante el desarrollo de la guía. **GitHub** es una plataforma ampliamente utilizada en la industria del software para gestionar proyectos, controlar versiones y facilitar la colaboración entre desarrolladores.

– En Github –

En primer lugar, se debe acceder a GitHub y crear un nuevo repositorio utilizando el botón **New**. Durante este proceso se asigna un nombre al proyecto y se genera la estructura inicial donde se almacenarán los archivos del ejercicio. Una vez creado el repositorio, GitHub proporcionará la información necesaria para vincularlo con las herramientas de desarrollo.

Dashboard

Top repositories

Find a repository...

New

1 General

Owner *

Repository name *

ejercicioPython

ejercicioPython is available.

Great repository names are short and memorable. How about [fantastic-chainsaw?](#)

Description

Ejercicios Tipo Python

22 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

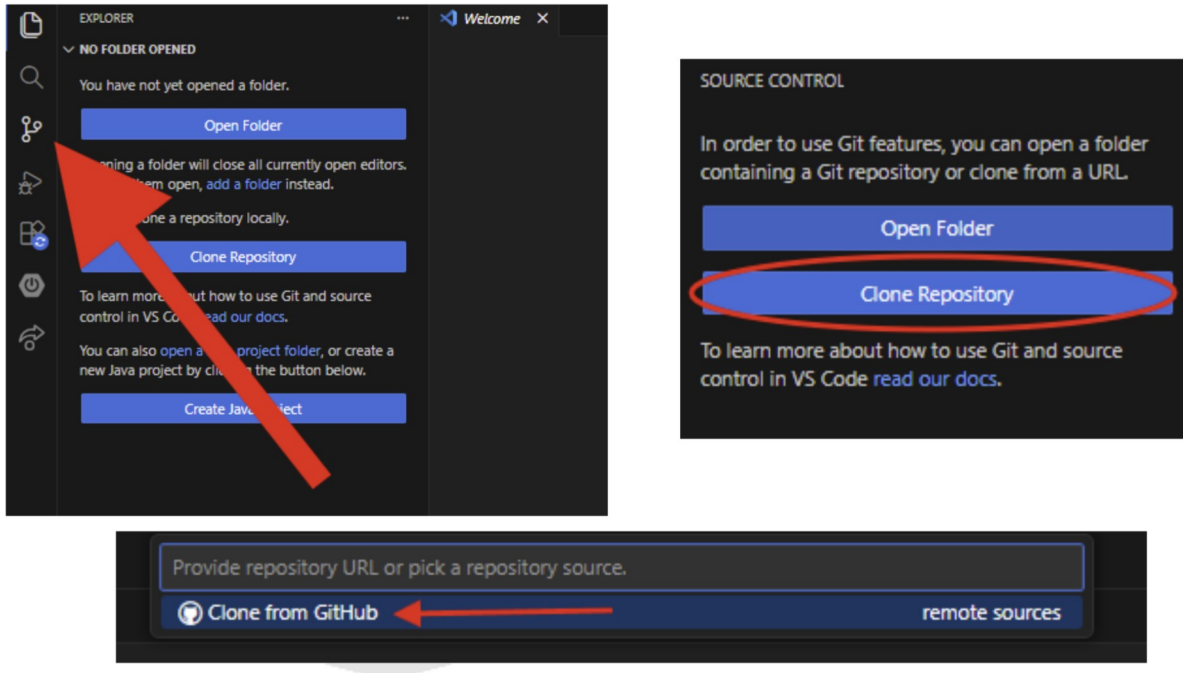
Licenses explain how others can use your code. [About licenses](#)

No license

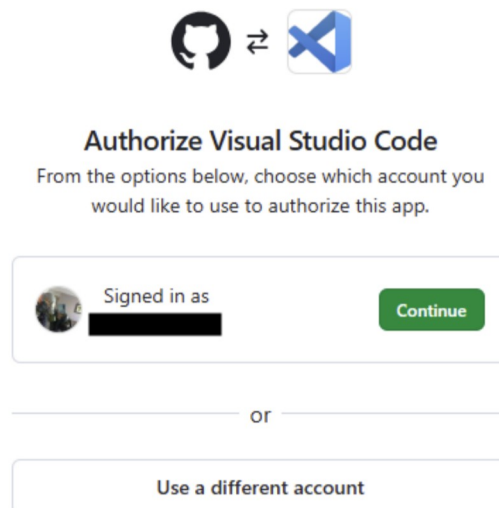
Create repository

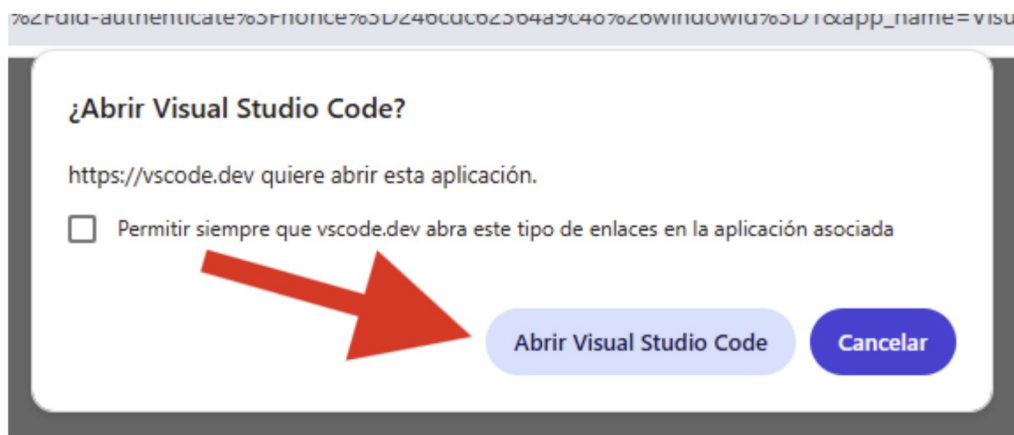
– En VSC –

Posteriormente, desde Visual Studio Code se debe iniciar sesión con la cuenta de GitHub para habilitar la sincronización entre el editor y la plataforma. Esta integración permite trabajar directamente sobre los repositorios sin necesidad de utilizar herramientas adicionales, simplificando el flujo de trabajo.



– En tu navegador de uso –



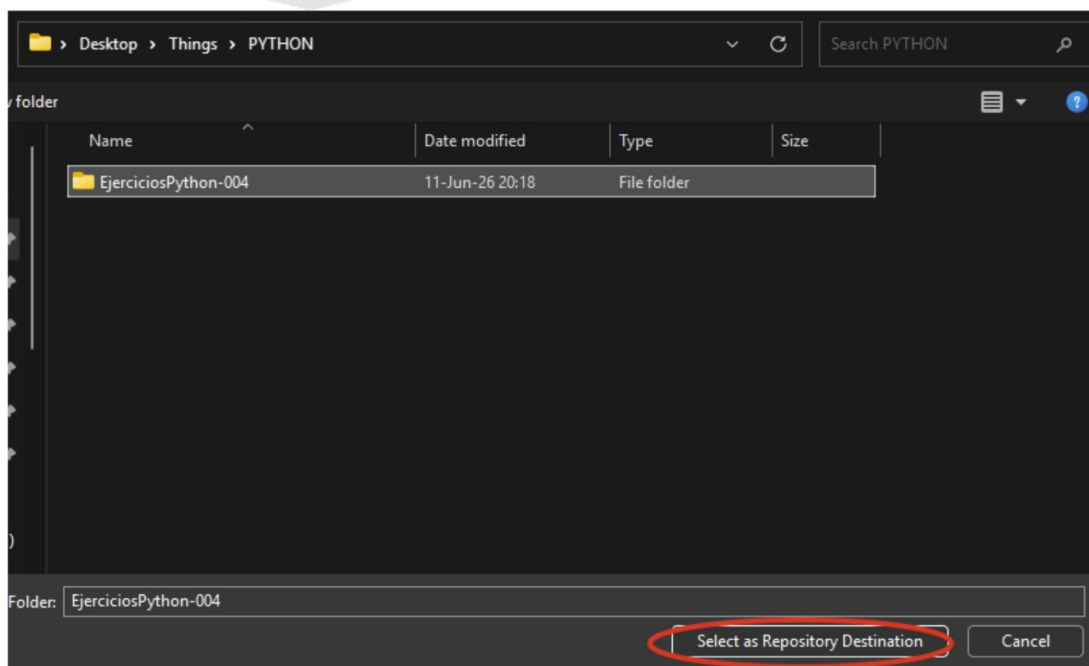


– En VSC –

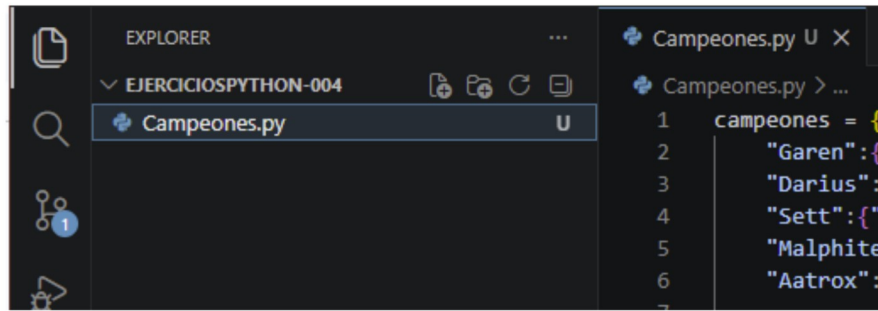
Una vez establecida la conexión, se procede a abrir o clonar el repositorio dentro de Visual Studio Code. Esto crea una copia local del proyecto en el computador, permitiendo agregar archivos, modificar código y organizar las carpetas necesarias para el desarrollo de los ejercicios.



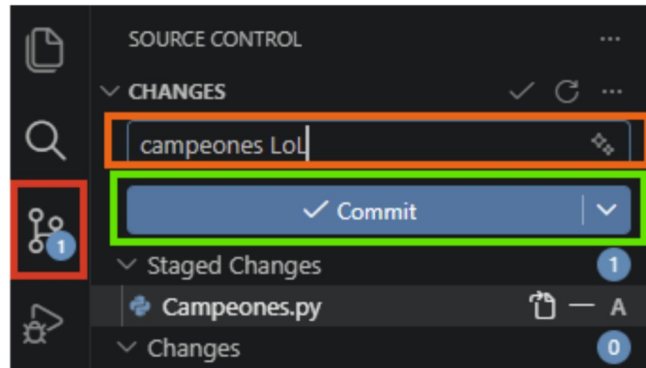
– En el buscador de archivos –



– En VSC –



Finalmente, cada vez que se complete una actividad, los cambios deberán ser guardados y enviados al repositorio remoto mediante una publicación o sincronización. De esta forma se garantiza que el trabajo quede respaldado en línea y disponible para su revisión, evaluación y seguimiento por parte del profesor.



– En Github –



Ejercicio 1: Administración de Propietarios (Fácil)

– Contexto –

La administración de un edificio residencial necesita mantener un registro actualizado de los propietarios de cada departamento. Para organizar la información, cada piso del edificio se almacenará en un diccionario independiente, donde la clave corresponderá al número del departamento y el valor será el nombre de su propietario. De esta forma, será posible consultar, modificar y administrar fácilmente los registros de los residentes.

Como asistente de la administración, deberás desarrollar un programa en Python que permita realizar distintas operaciones sobre estos registros, tales como consultar propietarios, actualizar información, agregar nuevos departamentos, eliminar registros y mostrar los datos almacenados. El objetivo es familiarizarse con el uso de diccionarios como estructura de almacenamiento y gestión de información dentro de un sistema simple.

– Objetivos –

Una comunidad de vecinos necesita registrar los propietarios de los departamentos de un edificio de 4 pisos.

Cada piso se almacenará en un diccionario independiente, donde:

- La clave será el número del departamento.
- El valor será el nombre del propietario.

Se dispone de la siguiente información inicial:

```
Piso1 = {  
    "101": "Juan Vélez",  
    "102": "María Soto"  
}  
  
Piso2 = {  
    "201": "Pedro Rojas",  
    "202": "Ana Díaz"  
}  
  
Piso3 = {  
    "301": "Carlos Pérez",  
    "302": "Claudia Torres"  
}  
  
Piso4 = {  
    "401": "José González",  
    "402": "Camila Muñoz"  
}
```

El programa deberá realizar las siguientes acciones:

1. **Mostrar** el propietario del departamento 202.
2. **Cambiar** el propietario del departamento 302 por el nombre "Juan Velez".
3. **Agregar** el departamento 103 con propietario "Pedro Sánchez" al Piso1.
4. **Eliminar** el departamento 402 del Piso4.
5. **Mostrar** todos los departamentos y propietarios del Piso2.
6. **Mostrar** la cantidad de departamentos registrados en el Piso1.
7. **Mostrar** solamente los nombres de los propietarios registrados en el Piso3.

– *Resultado Esperado* –

```
Dueño departamento 202: Ana Díaz

Piso3 actualizado:
301 : Carlos Pérez
302 : Juan Velez

Piso1 actualizado:
101 : Juan Vélez
102 : María Soto
103 : Pedro Sánchez

Piso4 actualizado:
401 : José González

Departamentos Piso2:
201 : Pedro Rojas
202 : Ana Díaz

Cantidad de departamentos en Piso1: 3

Propietarios Piso3:
Carlos Pérez
Juan Velez
```

Ejercicio 2: Pokédex de Movimientos (Medio)

– Contexto –

El Profesor Oak necesita un sistema para registrar los movimientos que conoce cada Pokémon. Para ello, **se utilizará un diccionario** donde:

- La clave será el nombre del Pokémon.
- El valor será una lista con los movimientos que puede utilizar.

Como ayudante del laboratorio, deberás desarrollar funciones para administrar la información de la Pokédex. **Debes:**

- Utilizar funciones para todas las operaciones.
- Utilizar un **diccionario** donde los valores sean listas.
- Todas las modificaciones deben realizarse directamente sobre la estructura entregada.

– Datos iniciales –

```
pokemon = {  
    "Pikachu": ["Impactrueno", "Ataque Rapido"],  
    "Charmander": ["Ascuas", "Arañazo"],  
    "Squirtle": ["Pistola Agua", "Placaje"]  
}
```

– Objetivos –

1. Mostrar Pokédex

Crear una función que deberá mostrar todos los Pokémon junto con sus movimientos.

– Ejemplo de salida –

```
Pikachu  
- Impactrueno  
- Ataque Rapido  
  
Charmander  
- Ascuas  
- Arañazo
```

2. Agregar movimiento

Crear una función que deberá solicitar:

- Nombre del **Pokémon**.
- Nombre del **movimiento**.

Luego deberá agregar el movimiento a la lista correspondiente.

– Ejemplo –

```
Pokémon: Pikachu
Movimiento: Cola Férrea
```

3. Mostrar movimientos de un Pokémon

Crear una función que deberá solicitar un Pokémon y mostrar todos sus movimientos.

– Ejemplo –

```
Pokémon: Squirtle
Movimientos:
- Pistola Agua
- Placaje
```

4. Contar movimientos

Crear una función que deberá **solicitar un Pokémon y mostrar cuántos movimientos tiene** registrados.

– Ejemplo –

```
Pikachu posee 2 movimientos.
```

5. Agregar un nuevo Pokémon

Crear una función que **deberá solicitar**:

- **Nombre del Pokémon.**
- **Primer movimiento.**

Luego deberá crear una nueva entrada en el diccionario.

– Ejemplo –

```
Pokémon: Bulbasaur
Movimiento: Látigo Cepa
```

– Ejemplo de Salida –

```
"Bulbasaur": ["Látigo Cepa"]
```

– 6. Eliminar un movimiento –

Crear una función que **deberá solicitar**:

- **Nombre del Pokémon.**
- **Movimiento a eliminar.**

Luego deberá eliminarlo de la lista.

Ejercicio 3: Registro de Videojuegos (Medio)

– Objetivos –

Desarrollar un programa en Python que simule un sistema de gestión de videojuegos para la tienda **Pixel Store**.

El sistema deberá permitir:

- **Registrar** videojuegos.
- **Buscar** videojuegos mediante su código.
- **Eliminar** videojuegos registrados.
- **Mostrar** el catálogo completo de videojuegos.
- **Mantener el sistema en ejecución** mediante un menú interactivo hasta seleccionar la opción de salida.

– Consideraciones –

Cada videojuego deberá almacenar la siguiente información:

- **Código**
- **Nombre**
- **Género**



La información deberá almacenarse utilizando un **diccionario**. Cabe mencionar que el **código de cada videojuego será único** y servirá como identificador principal. Finalmente, las operaciones de **agregar** y **eliminar** deberán **modificar** correctamente el catálogo.

– El sistema deberá validar–

- **Ingreso** de opciones numéricas.
- **Códigos no repetidos** al registrar videojuegos.
- **Que el código ingresado exista al buscar un videojuego.**
- **Que el código ingresado exista al eliminar un videojuego.**
- **Opciones válidas** dentro del menú.
- **Errores** de ingreso de datos mediante `try/except`.

– Ejemplos Consola –

– Menú Principal –

```
===== PIXEL STORE =====  
1. Registrar videojuego  
2. Buscar videojuego  
3. Eliminar videojuego  
4. Mostrar catálogo  
5. Salir  
Seleccione una opción: 
```

– Opción 1: Agregar videojuego –

```
Seleccione una opción: 1  
Ingrese código: 007  
Ingrese nombre: Devil May Cry  
Ingrese género: Action  
Videojuego agregado correctamente.
```

– Opción 2: Buscar videojuego –

```
Seleccione una opción: 2  
Ingrese código a buscar: 007  
  
Código: 007  
Nombre: Devil May Cry  
Género: Action
```

– Opción 3: Eliminar videojuego –

```
Seleccione una opción: 3  
Ingrese código a eliminar: 007  
Videojuego eliminado.
```

– Opción 4: Ver Catálogo –

– Objetos de Prueba Nuevos –

```
Código: 002  
Nombre: Fortnite  
Género: BattleRoyale
```

```
Código: 009  
Nombre: Metal Gear Solid 3: Snake Eater  
Género: Sigilo-Acción
```

– Catálogo –

```
Seleccione una opción: 4  
  
===== CATÁLOGO =====  
  
Código: 009  
Nombre: Metal Gear Solid 3: Snake Eater  
Género: Sigilo-Acción  
  
Código: 002  
Nombre: Fortnite  
Género: BattleRoyale
```

– Opción 5: Salir –

```
Seleccione una opción: 5  
Programa finalizado.
```

Ejercicio 4: Team Builder de LoL (Difícil)

– Contexto –

La organización de esports **Summoner Academy** se encuentra preparando la final de su torneo interno de League of Legends. Para analizar las probabilidades de victoria de cada equipo, los entrenadores han recopilado estadísticas de distintos campeones, incluyendo los roles que pueden desempeñar y su winrate promedio en partidas competitivas.

Tu tarea será desarrollar un sistema que permita formar dos equipos de cinco campeones respetando una composición estándar de League of Legends: un campeón para la línea superior (TOP), uno para la jungla (JG), uno para la línea central (MID) y dos campeones para la línea inferior (BOTTOM). Cada campeón sólo podrá ser seleccionado una vez y deberá ser capaz de desempeñar el rol asignado.

Una vez creados ambos equipos, el sistema deberá calcular el winrate promedio de cada uno utilizando las estadísticas disponibles. De esta manera, los entrenadores podrán determinar qué equipo tiene mayores probabilidades de ganar antes de que comience el enfrentamiento y analizar cuáles son las composiciones más fuertes para el torneo.

Además, debido a las reglas del torneo, los equipos no pueden compartir campeones y todas las selecciones deben ser validadas antes de comenzar la partida. Los analistas de **Summoner Academy** utilizarán esta herramienta para comparar distintas estrategias, evaluar el impacto de cada campeón en la composición final y descubrir qué combinación de jugadores tiene las mejores posibilidades de alcanzar la victoria en la Grieta del Invocador.

Campeón	Roles	Winrate (%)
Garen	TOP	51
Darius	TOP	52
Sett	TOP, JG	50
Malphite	TOP	53
Aatrox	TOP	49
Lee Sin	JG	49
Vi	JG	52
Jarvan IV	JG	51
Viego	JG	50
Wukong	TOP, JG	52
Ahri	MID	52
Yasuo	MID, TOP	49
Zed	MID	50
Veigar	MID	53
Lux	MID, BOTTOM	51
Jinx	BOTTOM	52
Vayne	TOP, BOTTOM	56
Ashe	BOTTOM	51
Caitlyn	BOTTOM	53
Miss Fortune	BOTTOM	52
Thresh	BOTTOM	51
Nautilus	BOTTOM	52
Leona	BOTTOM	50
Morgana	MID, BOTTOM	51
Pyke	MID, BOTTOM	53

– Objetivos –

Desarrollar un programa en Python que simule un sistema de creación de equipos para un torneo de League of Legends.

El programa debe utilizar obligatoriamente diccionarios, listas, funciones, ciclos y validaciones para gestionar correctamente la creación de equipos y el cálculo del ganador probable.

El sistema deberá permitir:

- **Crear** el Equipo Azul.
- **Crear** el Equipo Rojo.
- **Mostrar** los **equipos** registrados.
- **Calcular el ganador probable** según el winrate promedio de cada equipo.
- Mantener el sistema en ejecución mediante un menú interactivo hasta seleccionar la opción de salida.

Usted va a disponer de un diccionario con información de distintos campeones de League of Legends.

– Consideraciones –

Cada campeón almacena:

- **Nombre** del campeón (llave principal del diccionario).
- **Roles** que puede desempeñar.
- **Winrate** promedio.

– Ejemplo –

```
"Garen":{  
    "roles":["TOP"],  
    "winrate":51  
}
```

Para formar un equipo se deberán seleccionar exactamente 5 campeones respetando la siguiente composición:

- **1 TOP**
- **1 JG**
- **1 MID**
- **2 BOTTOM**

Cada campeón podrá ser utilizado una sola vez durante toda la partida, por lo que un campeón seleccionado en el Equipo Azul no podrá ser utilizado en el Equipo Rojo. **La información de los equipos deberá almacenarse utilizando diccionarios.**

Finalmente, el sistema deberá calcular el promedio de winrate de cada equipo para determinar cuál tiene mayores probabilidades de victoria.

– Cálculo del ganador –

El programa deberá obtener el promedio de winrate de cada equipo utilizando la siguiente fórmula:

- **Promedio = Suma de winrates / 5**

Posteriormente deberá mostrar:

- **Promedio Equipo Azul: XX.X%**
- **Promedio Equipo Rojo: XX.X%**

Con esta información usted debe determinar cuál de los dos equipos tiene más probabilidades de ganar.

– El sistema deberá validar –

- **Ingreso** de opciones numéricas.
- **Que el campeón ingresado exista** en el diccionario.
- **Que el campeón pueda desempeñar el rol solicitado.**
- **Que un campeón no sea seleccionado más de una vez.**
- **Que ambos equipos tengan exactamente:**
 - 1 TOP
 - 1 JG (Jungla)
 - 1 MID
 - 2 BOTTOM
- **Que existan ambos equipos** antes de calcular el ganador.
- **Opciones válidas** dentro del menú.
- **Errores** de ingreso de datos mediante `try/except`.

– Bonus: MVP –

Agregar una opción que permita mostrar el campeón con mayor winrate entre todos los seleccionados para ambos equipos.

– Ejemplo MVP –

```
===== MVP ESTADÍSTICO =====  
  
Wayne  
Winrate: 56%
```

– Ejemplos Consola –

– Menú Principal –

```
===== TEAM BUILDER =====  
1.- Crear Equipo Azul  
2.- Crear Equipo Rojo  
3.- Mostrar Equipos  
4.- Calcular Ganador Probable  
5.- Salir
```

– Opción 1: Crear equipo Azul –

```
Ingrese opción: 1  
  
===== CREANDO EQUIPO AZUL =====  
Ingrese campeón para TOP: Garen  
Ingrese campeón para JG: Lee Sin  
Ingrese campeón para MID: Zed  
Ingrese campeón para BOTTOM: Jinx  
Ingrese campeón para BOTTOM: Ashe
```

– Opción 2: Crear equipo Rojo –

```
Ingrese opción: 2  
  
===== CREANDO EQUIPO ROJO =====  
Ingrese campeón para TOP: Wukong  
Ingrese campeón para JG: Vi  
Ingrese campeón para MID: Yasuo  
Ingrese campeón para BOTTOM: Thresh  
Ingrese campeón para BOTTOM: Pyke
```

– Opción 3: Mostrar equipos –

```
Ingrese opción: 3  
  
===== EQUIPO AZUL =====  
TOP: Garen  
JG: Lee Sin  
MID: Zed  
BOTTOM1: Jinx  
BOTTOM2: Ashe  
  
===== EQUIPO ROJO =====  
TOP: Wukong  
JG: Vi  
MID: Yasuo  
BOTTOM1: Thresh  
BOTTOM2: Pyke
```

– Opción 4: Calcular Equipo probable –

```
Ingrese opción: 4  
  
Promedio Equipo Azul: 50.6%  
Promedio Equipo Rojo: 51.4%  
El Equipo Rojo tiene mayores probabilidades de ganar.
```

– Opción 5: Salir–

```
Ingrese opción: 5  
Programa terminado...
```

Ejercicio 5: Registro Planetario (Muy Difícil)

– Contexto –

En el milenio 41, la Super Tierra continúa expandiendo su influencia a través de miles de sistemas estelares. Cada nuevo mundo descubierto debe ser registrado y clasificado según sus características demográficas para garantizar una correcta administración de recursos, defensa y producción.

La Oficina de Colonización Planetaria ha solicitado el desarrollo de un sistema que permita almacenar información sobre los planetas bajo control de la Super Tierra. **Cada colonia deberá contar con un código único, su nombre, cantidad de habitantes y una clasificación planetaria.** Dependiendo de su población, los mundos pueden convertirse en enormes centros urbanos, importantes polos industriales, regiones agrícolas, territorios hostiles o incluso planetas completamente deshabitados.

La siguiente tabla muestra los tipos de mundos reconocidos oficialmente y los rangos poblacionales asociados a cada uno de ellos:

Tipo	Descripción del planeta	Cantidad Min	Cantidad Máx
Colmena	Planetas-ciudad masivamente superpoblados, caracterizados por imponentes torres y niveles inferiores.	100.000.001	$\infty+$
Forja	Dedicados exclusivamente a la manufactura, cubiertos por completo de maquinaria industrial.	100.000.001	$\infty+$
Agrícolas	Planetas rurales utilizados exclusivamente para producir alimentos para el resto de los sistemas.	100.000.001	$\infty+$
Mortíferos	Planetas con entornos extremadamente hostiles, a menudo letales, y ecosistemas agresivos.	1	100.000.000
Salvajes	Mundos preindustriales o de nivel medieval.	1	100.000.000
Muertos	Planetas estériles, desolados o destruidos.	0	0

Deberás desarrollar una aplicación en Python que permita registrar, consultar, actualizar y eliminar información de las colonias registradas, asegurando que cada planeta sea clasificado correctamente según las reglas establecidas por la administración central de la Super Tierra.

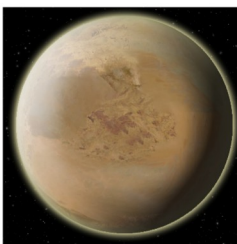
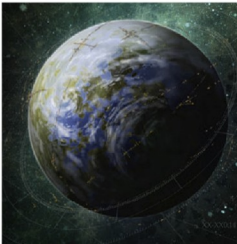
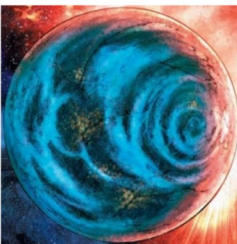
Como funcionario de la Oficina de Colonización Planetaria, tu deber es mantener registros precisos y actualizados para garantizar que cada planeta contribuya al bienestar colectivo.

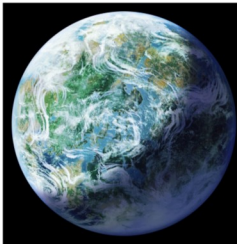
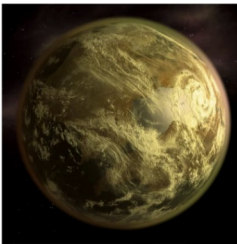
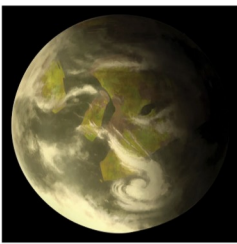
“Ser el primero en entrar en el cosmos, entablar, en solitario, un duelo sin precedentes con la naturaleza... ¿se puede soñar con algo más?”

– Yuri Gagarin, el primer ser humano en el espacio. –

– Ejemplos de planetas colonizados –

				
1000	1001	1002	1003	1003
Super Tierra	Necromunda	Catachan	Coruscant	Tatooine
Colmena	Colmena	Mortífero	Colmena	Agrícolas
1.892.092.293.221	19.092.595.220	620.201	990.925.932.213	320.123.155

				
1004	1005	1006	1007	1008
Arrakis	Macragge	Cybertron	Demerium	Krypton
Mortífero	Colmena	Forja	Muerto	Muerto
62.340.023	1.209.259.322.132	110.289.323	0	0

				
1009	1010	1011	1012	1013
Endor	Helghan	Marte	Felucia	Cadia
Salvajes	Forja	Forja	Agrícolas	Muerto
				0

– Objetivos –

Desarrollar un programa que permita administrar el registro de mundos colonizados por la **Super Tierra**

El sistema deberá permitir:

- **Registrar** nuevos planetas.
- **Buscar** información de un planeta.
- **Actualizar** la población de un planeta.
- **Eliminar** planetas del registro.
- **Mostrar** todas las colonias registradas.
- **Mantener el sistema en ejecución mediante un menú interactivo** hasta seleccionar la opción de salida.

– Consideraciones –

Cada planeta deberá almacenar la siguiente información:

- **Código** planetario.
- **Nombre** del planeta.
- **Población**.
- **Tipo de Planeta**.

Los datos deberán almacenarse utilizando diccionarios en python y el código planetario será único para cada planeta.

Los tipos de mundos válidos serán:

- **Colmena**
- **Forja**
- **Agrícola**
- **Mortífero**
- **Muerto**
- **Salvaje**

Los tipos de planeta deberán almacenarse en una lista.

La población determinará los posibles tipos de planeta que pueden ser asignados:

Tipo Planeta	Cantidad Min	Cantidad Máx
Colmena	100.000.001	$\infty+$
Forja	100.000.001	$\infty+$
Agrícolas	100.000.001	$\infty+$
Mortíferos	1	100.000.000
Salvajes	1	100.000.000
Muertos	0	0

- Si la población es igual a 0, el tipo de planeta deberá asignarse **automáticamente como "Muerto"**.
- Si la población está entre 1 y 100.000.000 habitantes, el usuario deberá seleccionar entre los tipos **"Mortífero"** y **"Salvaje"**.
- Si la población es superior a 100.000.000 habitantes, el usuario deberá seleccionar entre los tipos **"Colmena"**, **"Forja"** y **"Agrícola"**.

Al actualizar la población de un planeta, el tipo deberá de re-calcularse y actualizarse según las reglas anteriores.

– El sistema deberá validar –

- Ingreso de opciones numéricas.
- Códigos planetarios no repetidos.
- Poblaciones mayores o iguales a 0.
- Existencia del planeta al buscar, modificar y eliminar.
- Tipos válidos de planeta.
- Opciones válidas dentro del menú.
- Errores de ingreso de datos mediante `try/except`.

– Ejemplos Consola –

– Menú Principal –

```
===== SUPER TIERRA =====
1. Registrar planeta
2. Buscar planeta
3. Actualizar población
4. Eliminar planeta
5. Mostrar colonias
6. Salir
Seleccione una opción: 
```

– Opción 1: Registrar Planeta –

```
Seleccione una opción: 1
Ingrese código planetario: 1001
Ingrese nombre del planeta: Super Tierra
Ingrese población: 123123123123
```

```
La población permite los siguientes tipos:
1. Colmena
2. Forja
3. Agrícola
Seleccione tipo: 1
Planeta registrado con éxito.
```

– Planeta Muerto –

```
Ingrese código planetario: 1012
Ingrese nombre del planeta: Demerium
Ingrese población: 0
Tipo asignado automáticamente: Muerto
Planeta registrado con éxito.
```

– Opción 2: Buscar Planeta –

```
Seleccione una opción: 2
Ingrese código del planeta: 1001

===== INFORMACIÓN DEL PLANETA =====
Código: 1001
Nombre: Super Tierra
Tipo: Colmena
Población: 123123123123
```

– Opción 3: Actualizar Población –

```
Seleccione una opción: 3
Ingrese código del planeta: 1001
Ingrese nueva población: 1234567890
```

```
La población permite los siguientes tipos:
1. Colmena
2. Forja
3. Agrícola
Seleccione tipo: 3
Población actualizada correctamente.
```

```
===== INFORMACION DEL PLANETA =====
Código: 1001
Nombre: Super Tierra
Tipo: Agrícola
Población: 1234567890
```

– Opción 4: Eliminar Planeta –

```
Seleccione una opción: 4
Ingrese código del planeta: 1001
Planeta eliminado.
```

– Opción 5: Mostrar Colonias –

– Objetos de ejemplo –

```
Código: 1001  
Nombre: Super Tierra  
Tipo: Colmena  
Población: 1892092293221
```

```
Código: 1002  
Nombre: Catachan  
Tipo: Mortífero  
Población: 620201
```

```
Código: 1012  
Nombre: Demerium  
Tipo: Muerto  
Población: 0
```

– Mostrar Colonias –

```
Seleccione una opción: 5  
  
===== COLONIAS REGISTRADAS =====  
  
Código: 1001  
Nombre: Super Tierra  
Tipo: Colmena  
Población: 1892092293221  
  
Código: 1002  
Nombre: Catachan  
Tipo: Mortífero  
Población: 620201  
  
Código: 1012  
Nombre: Demerium  
Tipo: Muerto  
Población: 0
```

– Opción 6: Salir –

```
===== SUPER TIERRA =====  
1. Registrar planeta  
2. Buscar planeta  
3. Actualizar población  
4. Eliminar planeta  
5. Mostrar colonias  
6. Salir  
Seleccione una opción: 6  
Programa finalizado.
```

¡Ánimo Chicos!

¡Cualquier consulta, reclamo, queja, duda que tengan no duden en contactarme!
¡Mándame tu ejercicio y yo te lo reviso!

Correo Institucional: mi.gonzalez@duocuc.cl

